Rupa

Taking the markup out of Claude's documents, and measuring what that saves

VAJRA SARPA · OCTOBER 2026

The problem

Ask Claude Code for a PDF and it does what a careful person would: writes HTML and CSS, finds a PDF tool, renders, looks at the result, and fixes what's wrong. It works. It's also expensive in a way that's easy to miss.

An agent session isn't billed per answer; it's billed per turn, and every turn re-reads the whole conversation so far. A document that takes twenty turns doesn't cost twenty small steps. It costs twenty readings of an ever-growing transcript, full of markup Claude wrote, rendered pages it looked at, and fixes to both. Most of what Claude wrote wasn't content. It was form.

I wanted to know how much of that could go, and I wanted to measure it rather than guess.

The idea: split content from form

Rupa (*rūpa* is Sanskrit for "form") is a small command-line tool and a Claude Code plugin built on one rule: Claude writes the content, Rupa owns the form.

Claude writes plain Markdown with a little frontmatter (title, subtitle, author, date, theme, an optional page budget). Rupa renders it through a theme into HTML or PDF, offline, with bundled fonts so it looks the same on every machine. Everything visual lives in the theme. Claude never writes a line of CSS, and if a document looks wrong, the fix is in the Markdown or the theme, never in a one-off style.

The contract between Claude and Rupa is deliberately narrow:

- **One status line out, never the document.** `rupa: wrote report.pdf (pdf, 3 pages, 41,903 bytes)`. Claude doesn't need to read back what it made to know it worked.
- **Exit codes that say what to do.** `3` is bad input, `4` a render failure, `5` a failed check or a PDF still over its page budget, with the status line saying how many lines to cut.
- **No raw HTML, no network.** Raw HTML in the Markdown is escaped, images must be local files, and a missing image fails the render instead of vanishing.

A narrow contract is what makes the savings possible. It's also what the plugin's evals test, so it can't drift.

Checking the result without a model in the loop

A renderer that only promises the file exists isn't much help. Rupa also checks its own output, in two tiers.

The first tier is code. `rupa verify` reads the PDF and runs eight checks: text colour, page size, page count, blank, near-empty and sparse pages, headings stranded at the bottom of a page, and anything running into the margin. Each check reports exactly what and where, such as `p1 table by 13pt`.

The second tier is visual, and it's the part I'm proudest of. For every page a check flags, verify writes a sheet: the theme's reference page on the left and the rendered page on the right. Claude, which is already a vision model and already in the session, judges the sheet against a short rubric and fixes the Markdown. Rupa itself never calls a model. A clean PDF gets no sheets at all, so the visual review costs nothing when nothing is wrong.

Measuring it

The benchmark gives `claude -p` the same prompt in a fresh, empty folder, so no run can see the repository, across four arms: Claude Code with no plugin, Rupa render-only, Rupa with a review sheet for every page type, and Rupa as shipped. Three runs per arm, on Claude Opus 5.5, with costs as Claude Code reports them at API list prices. A script reads each run's transcript for turns, tokens, time and every tool call, so I could see why a number moved, not just that it did.

The first benchmark said no

The first result was humbling. On a two-page report, Rupa didn't save anything. The render-only arm took 11% more turns than plain Claude Code, and the arm with per-type review took 67% more.

The transcripts showed why, and it wasn't rendering:

- **Every Rupa run overshot two pages on its first render.** Plain Claude Code fixed the same overshoot in one turn by editing its own CSS. Rupa's form is fixed, so Claude cut content instead, probing page counts with `pdftotext | wc -w`. One run took seven cut-and-render rounds.
- **The review arm threw its saving away.** When verify made no sheets, Claude rendered the pages itself and looked anyway.

The fixes went into the system, not into a cleverer prompt. Rupa gained page budgets: given `pages: 2`, it fits the PDF itself by dropping the contents page and tightening type, and only when that isn't enough does it say how much to cut. Review sheets became flagged-only by default, and the skill says plainly not to look when there are none.

On the next run, every Rupa document fitted on its first render. The arm that verifies went from 67% more turns than plain Claude Code to 40% fewer.

What the measurements found next

Each new benchmark task exposed one more place where Claude was doing work the tool should do.

Cut hints in the wrong unit. On a three-page technical guide, runs that had to cut still took two to four rounds. Rupa said "cut about 34 words", but the overflow was a JSON policy at one or two words a line, so 34 words meant fifteen lines of space. The hint now measures height and speaks in lines, and the skill makes each cut with a single rewrite of the file rather than a string of small edits. On a task built to force cutting, every run that went over cut once and fitted.

A table too wide for the page. The hardest task asked for an eleven-column table. Every Rupa first draft overflowed, and the shipped plugin saved only 13%: Claude kept shortening cell text, round after round, and the table stayed 13 points too wide. Verify had only said "text extends past the side margins". It now names the culprit and the distance, and the theme sets tables of nine or more columns in smaller type. After that change, every run fitted on its first render, and the saving went from 13% to 40%.

Plain Claude Code handled the same table differently, and not badly. It turned the page to landscape or shrank the type to 6.6 points, breaking one header mid-word. Rupa won't do either, because both break the theme. That's a real trade-off, and it's why the fix belonged in the theme.

What the machines disagreed about

Once the look was worth protecting, I added a golden-render test to CI. It renders a sample document with every theme and compares each page to a committed reference image. Locally the comparison was exact.

On CI it failed, on one page only: code. Prose, tables and the cover came out pixel-identical between Fedora and Ubuntu; the code page was 1.9% off. Each code line's height came from mixing two fonts' metrics, the body font's line box around the code font's glyphs, and the two systems' text libraries read those metrics slightly differently. A second bug hid behind it: the syntax highlighter's stylesheet loaded after the theme and overrode its line height. Code blocks now carry their own font and a fixed line height

at every fit level, and the print rules load last. The rule I took from it: never let a different font's metrics decide a line's height.

What real use found

Benchmarks only test what you think to ask. Using the installed plugin on everyday documents found three things they hadn't:

- **Markdown people type without thinking printed literally.** Task lists, strikethrough and footnotes came out as `[x]`, `~~old~~` and `[^1]`. Verify couldn't see it, because nothing was out of place. They're enabled now, still with raw HTML escaped.
- **Verify was blind to images.** A heading, its one-line lead-in and a chart at the end of a page read as a stranded heading, and the page measured half full when it was nearly full. Images now count.
- **HTML linked its images instead of embedding them,** so a sent file lost its chart. HTML output now carries its images inside the file.

Results

On the three benchmark tasks, Claude Code with Rupa against Claude Code alone (medians of three runs each):

	Claude Code alone	With Rupa	Gain
Cost per document	$0.52 – $1.19	$0.29 – $0.33	40–74% cheaper
Time per document	112 – 252 s	36 – 52 s	2.4–5.6× faster
Context added per document	12,200 – 34,300 tokens	3,800 – 5,800 tokens	61–86% less
PDFs passing `rupa verify`	0 of 9	9 of 9	

The three gains share one cause. Claude stops writing and re-reading markup, and Rupa answers in one line. The context figure matters beyond cost, because that context stays for the rest of the session: five guides would fill about 17% of the model's million-token window without Rupa, and 2.4% with it.

The harder the layout, the bigger the gap. On the three-page guide, plain Claude Code took 23 turns, $1.19 and over four minutes; Rupa took 6 turns, $0.31 and 45 seconds.

What it doesn't do

- **It only runs in Claude Code.** Rupa is a local tool plus a plugin, so the Claude chat app can't use it, and I haven't measured the chat app.
- **It makes HTML and PDF only.** Slides are the natural next format; DOCX would need much heavier tooling to check.
- **Looks come from themes.** That's the point, but it means a new look is CSS work. I built two themes from my own slide-deck design system, one with a dark PDF and one that prints black on white.
- **Part of the saving is less writing.** At a fixed page count, Rupa keeps the theme's margins, so its two-page report held about 850 words against about 1,180. The baseline PDFs also fail verify partly by Rupa's own rules, such as pure black text, though they were readable.
- **There's a floor.** About $0.19 of every run is Claude Code starting a session, so short, simple documents save least.
- **The sample is small.** Three runs per arm, one model, my own tasks. Agent runs vary; the medians matter more than any single run.

What I took from it

Measure the transcript, not just the output. Every improvement here came from reading why runs took the turns they did. The cost total only said that something was wrong; the tool calls said what.

Make the tool's answers small. One status line instead of the document, and sheets only when something is flagged. The cheapest token is the one never put in the conversation.

Fix the system, not the prompt. Page budgets, line-based cut hints, a named overflow and a theme rule for wide tables each did more than any instruction could, because they removed work instead of describing it better.

Rupa is open source under the MIT licence, with the benchmark, its results and a 93-page build guide in the repository. I built it with Claude Code, which made the measurements an unusually honest check: the agent the tool is for did the work being measured.